

Computational Complexity

P, NP, co-NP, Reductions, NP-hard & NP-complete

A from-zero-to-advanced study guide for CSE 207 · Lecture slides 1-28

What is inside

- Every idea in slides 1-28 explained **from scratch** in plain language, in the same order as the slides, but with extra background so nothing feels like a jump.
- **Every question the slides ask** (“Do you see why...?”, “How?”, “Is it true that...?”) is answered in an **orange box**.
- Coloured boxes: **blue** = key idea, gray = example, **red** = common mistake, **green** = remember this.
- A one-page **cheat sheet** for revision before the test.
- **33 CT-style questions** ordered Easy → Hard (Part 11), with full worked answers (Part 12). Try each question first, then check the answer.

Reading map

Part	Topic	Slides
0	Warm-up: problems, instances, polynomial vs exponential	(background)
1	The big picture: why we cannot just “find a better algorithm”	2-4
2	Decision problems and CNF/SAT	5
3	Optimization vs. decision (they are equally hard)	8
4	The classes P and NP, certificates, verification	6-7
5	$P \subseteq NP$ and the P vs NP question	9-11
6	Complement of a problem, and co-NP	12-16
7	NP vs co-NP and how P, NP, co-NP are related	17-21
8	Polynomial-time reductions	22-24
9	NP-hard, NP-complete, and how to prove NP-completeness	25-28
10	Cheat sheet	—
11	CT practice questions (Easy → Hard)	—
12	Answers to practice questions	—

Part 0 · Warm-up: the words you need

0.1 Problem vs. instance

A **problem** is a general question, e.g. “Given a list of numbers, sort it.” An **instance** is one concrete input, e.g. the list [3, 1, 2]. Algorithms solve *problems*; they are run on *instances*. This matters later: a **reduction** is a machine that turns an *instance* of one problem into an *instance* of another.

0.2 Input size and running time

We measure an algorithm by how many basic steps it takes as a function of the input size **n** (number of elements, vertices, bits ...). We care about how the count *grows* when *n* gets big.

0.3 Polynomial vs. exponential: the definition of “efficient”

An algorithm is **polynomial time** if its running time is $O(n^k)$ for some fixed constant *k* (n , $n \log n$, n^2 , n^3 , n^{50} ...). In this course, **efficient = polynomial time** (slide 2: “ $O(n^k)$ time algorithm, that’s the norm”). An **exponential** running time looks like 2^n or $n!$ — the exponent contains *n*.

n	n^2	n^3	2^n
10	100	1,000	1,024
20	400	8,000	1,048,576
50	2,500	125,000	$\approx 1.1 \times 10^{15}$
100	10,000	1,000,000	$\approx 1.3 \times 10^{30}$
Time for $n = 100$ at 10^9 steps/sec	10^{-5} s	10^{-3} s	$\approx 4 \times 10^{13}$ years

The last cell is about 3,000 times the age of the universe. That is why the whole subject is about separating **polynomial** (“doable”) from **exponential** (“hopeless for big *n*”).

REMEMBER · Why polynomial?

Two reasons: (1) polynomials are **closed** under the operations we use to build algorithms — a polynomial number of steps, each costing a polynomial, is still polynomial; (2) in practice, problems with polynomial algorithms usually end up with small exponents (n , n^2 , n^3), while exponential ones become impossible quickly.

Part 1 · The big picture (slides 2-4)

1.1 Searching among exponentially many possibilities

Many problems ask us to pick the **best** solution out of an **exponentially large** set of candidates. A graph with *n* vertices has up to n^{n-2} different spanning trees, and up to $n!$ different tours through all vertices. Checking all of them is hopeless for large *n*.

Surprisingly, some of these problems *do* have fast algorithms, because of clever structure:

shortest path (Dijkstra, Bellman–Ford) and **minimum spanning tree** (Kruskal, Prim). The slide stresses: **these are the exceptions!** For a huge list of other problems (SAT, Traveling Salesman, Vertex Cover, Hamiltonian Cycle ...) nobody has found an efficient algorithm.

1.2 The cartoon: “I’m dumb” vs. “there isn’t any”

The slide (from Garey & Johnson’s classic book *Computers and Intractability*) shows an employee facing his boss. He has two possible excuses:

- “I cannot solve it efficiently **because I am dumb**” — bad: sounds like personal failure.
- “I cannot find an efficient solution **because there isn’t any**” — great, but you must *prove* it.

The slide’s question: *How do you know there is no efficient solution?* For some problems we can prove it. Example: any sorting algorithm that only **compares** elements needs $\Omega(n \log n)$ comparisons (there are $n!$ possible orderings, and each comparison at best halves the possibilities, so you need at least $\log_2(n!) \approx n \log n$ comparisons). That is a **lower bound**.

1.3 What do we do when we cannot prove a lower bound?

It is **really hard** to prove that a problem has no efficient algorithm (slide 4). For most problems in this course nobody can. So we do the next best thing:

KEY IDEA · Relative hardness

Show that our problem is **as hard as** many other problems that brilliant scientists have attacked for decades without finding an efficient algorithm. Then the boss can no longer say you are dumb — the whole scientific community is “dumb” too. Complexity theory sorts problems into **equivalence classes** and compares their **relative difficulty**. The tool for comparing is the **reduction** (Part 8). The result is the family of classes **P, NP, co-NP, NP-hard, NP-complete**.

Part 2 · Decision problems (slide 5)

2.1 Definition

A **decision problem** is a problem whose output is a single Boolean value: **YES or NO**. The theory of NP is built on decision problems because a yes/no answer is a clean, uniform thing to reason about. (Part 3 shows that we lose nothing by doing this.)

2.2 Turning an optimization problem into a decision problem

Optimization version (find the best)	Decision version (yes/no)
Shortest path: find the shortest path between s and t in G .	Is there a path P between s and t in G with length $p \leq x$?
Minimum spanning tree: find a spanning tree of least weight.	Is there a spanning tree of total weight $\leq W$?
Given a CNF formula: find a satisfying assignment.	Given a CNF formula: does <i>any</i> satisfying assignment exist?

SLIDE QUESTION (slide 5): Do you see why a bound ($p \leq x$) is required here?

A yes/no question needs a **threshold** to say yes or no to. Without the bound, “Is there a path between s and t ?” only tests **reachability** (a plain BFS answers it) and says nothing about shortness. The bound x encodes “good enough”: YES means “there is a path at least as short as x ”. By changing x we can home in on the optimum (Part 3). The same happens for every optimization problem: TSP needs “tour length $\leq L$ ”, Vertex Cover needs “size $\leq k$ ”, and so on.

2.3 Boolean formulas, CNF, and SAT (the star example)

- A **variable** takes value 1 (true) or 0 (false). A **literal** is a variable or its negation: X or X' (X' means NOT X).
- A **clause** is an OR ($\vee$) of literals: $(X \vee Y \vee Z)$. It is true if *at least one* literal is true.
- A formula is in **CNF** (conjunctive normal form) if it is an AND ($\wedge$) of clauses. It is true only if every clause is true.
- **SAT**: given a CNF formula, is there an assignment of 0/1 to the variables that makes it true? ("satisfiable?") **3-SAT** is SAT where each clause has exactly 3 literals.

EXAMPLE · the slide's formula

$(X \vee Y \vee Z) \wedge (X' \vee Y' \vee Z)$. Try $X = 1, Y = 0, Z = 1$. Clause 1: $X = 1 \checkmark$. Clause 2: $X' = 0, Y' = 1 \checkmark$. Both clauses true, so the formula is **satisfiable** and the answer is YES.

Part 3 · Optimization vs. decision (slide 8)

The slide claims: "Turning an optimization problem into a decision problem does not necessarily change its difficulty. They reduce to one another!" Two directions need explaining.

3.1 Optimization $\Rightarrow$ decision (easy)

SLIDE QUESTION (slide 8): "Solving the optimization problem readily solves the decision problem" — how?

Run the optimization algorithm, get the optimal value L (e.g. shortest path length), and just **compare**: answer YES iff $L \leq x$. One extra comparison, so the decision problem is no harder than the optimization problem.

3.2 Decision $\Rightarrow$ optimization (the trickier direction)

SLIDE QUESTION (slide 8): "You can also know the shortest path by using the solutions to the decision problem. Do you see how?"

Assume we have a black box $\text{DECIDE}(G, s, t, x) = \text{"is there an } s\text{-}t \text{ path of length } \leq x\text{"}$. Assume integer edge weights and let $W = \text{sum of all weights}$ (an upper bound on any simple path).

Step 1: find the optimal length L by binary search. Ask DECIDE with $x = W/2$; if YES, try smaller, else larger. The first x where the answer switches from NO to YES is L . That needs only about $\log_2(W)$ calls.

Step 2: find the actual path. Look at the edges one at a time:

```
for each edge e in G:
    remove e from G
    if DECIDE(G, s, t, L) == NO:      # e was necessary
        put e back
# the edges that are left form a shortest s-t path
```

Each edge is removed only if a path of length $\leq L$ still exists without it. At the end, every remaining edge is necessary, so what remains is exactly one shortest path. Total: polynomially many calls to the decision black box. Hence **optimization $\leq_p$ decision** as well, so the two versions are equally hard (up to polynomial factors).

REMEMBER · Why this justifies the whole approach

Since the decision version and the optimization version are polynomially equivalent, proving the decision version is hard (or easy) tells us the same about the optimization version. This is why we can study NP through yes/no questions only.

Part 4 · The classes P and NP (slides 6-7)

4.1 Class P: solved fast

P is the set of decision problems that can be **solved** in polynomial time. (Slide: “relatively easier problems that can be solved quickly!”) Examples: “Is there an s-t path of length $\leq x$?” (Dijkstra), “Is this list sorted?”, “Is n prime?” (proved in 2002, see Part 6), “Is there a spanning tree of weight $\leq W$?”.

4.2 Class NP: checked fast

NP is the set of decision problems for which a **YES** answer can be **verified** in polynomial time when someone hands us the answer (slide: “we can verify a YES answer if we have the solution in front of us”). Slogan from the slide: **Hard to find, but easy to recognize!**

EXAMPLE · Sudoku / jigsaw

Solving a hard Sudoku may take a long time. But if a friend hands you a completed grid, checking it (each row, column and box has 1-9) takes seconds. Sudoku-like puzzles are “hard to find, easy to recognize”.

Certificate and certifier: the formal language

The slide says: “Formally, given an input (instance) I and a solution S , we can efficiently check whether S indeed corresponds to the YES answer.” The words used in the course:

- **Instance I** — the input of the decision problem.
- **Certificate S** (also called *witness* or *proof*) — a proposed “reason” why the answer is YES. Its length must be polynomial in $|I|$.
- **Certifier (verifier) B** — an algorithm that gets (I, S) , runs in **polynomial time**, and answers accept/reject. For a YES instance there *exists* some S that B accepts; for a NO instance *no* S makes B accept.

KEY IDEA · NP in one sentence

$X \in \text{NP} \Leftrightarrow$ there is a polynomial-time certifier B with: I is a YES-instance $\Leftrightarrow$ **there exists** a certificate S such that $B(I, S)$ accepts.

WATCH OUT · NP does NOT mean “not polynomial”

NP = **N**ondeterministic **P**olynomial time (a hypothetical machine that *guesses* the certificate and then verifies it). Every problem in P is also in NP (Part 5), so NP cannot mean “not polynomial”.

4.3 The SAT example (slide 7)

Question: is the formula $I: (X \vee Y \vee Z) \wedge (X' \vee Y' \vee Z)$ satisfiable? The slide gives the certificate $S: X = 1, Y = 0, Z = 1$.

- **Verification** (polynomial): plug the values into each clause and evaluate. This takes time linear in the formula size. So we can check in polynomial time whether S satisfies I .
- **Solving from scratch:** nobody knows a polynomial algorithm. Brute force tries all 2^n assignments. The slide notes: “may not be in P (but nobody knows)”.
- Therefore **SAT $\in$ NP** (and we do not know whether SAT $\in$ P).

SLIDE QUESTION (slide 7): Do you see why there is no efficient algorithm to verify the “NO” answer?

To verify a YES we need to exhibit **one** assignment (“there exists” a satisfying assignment). To verify a NO (formula is **unsatisfiable**) we would have to show that **every one** of the 2^n assignments fails (“for all”). A single assignment proves nothing about the others, and nobody knows a short certificate that proves “no assignment works” for all formulas. Careful: this is “no *known* way”, not a proof that none exists. If a short NO-certificate existed for every formula, then NP = co-NP (Part 7), which is widely believed to be false.

Problem	Fast to solve? (in P)	Fast to verify a YES? (in NP)
Shortest path $\leq x$?	Yes (Dijkstra)	Yes: the path itself
SAT	Not known	Yes: a satisfying assignment
Vertex Cover of size $\leq k$?	Not known	Yes: the k vertices
Hamiltonian cycle?	Not known	Yes: the cycle
Sorted list?	Yes	Yes (trivially)

Part 5 · $P \subseteq NP$ and the P vs NP question (slides 9-11)

5.1 Every problem in P is in NP (slide 9)

KEY IDEA · P is a subset of NP

Claim: $X \in P \Rightarrow X \in NP$. **Proof.** Let A be a polynomial-time algorithm that solves X . Build a certifier $B(I, S)$ that **ignores the certificate S** , runs A on I , and returns A 's answer. B runs in polynomial time and accepts exactly the YES instances. So B is a valid polynomial certifier and $X \in NP$. ■

Intuition: if you can solve a problem fast, you do not need any hint to check the answer.

5.2 The million-dollar question (slide 10)

We know $P \subseteq NP$. The slide asks the two versions of the same question:

- Is $NP \subseteq P$? i.e. is **$P = NP$** ? (Can everything that is easy to check also be solved fast?)
- Or is there a problem in NP that does not belong to P? i.e. **$P \neq NP$** ?

This is the most famous open problem in computer science (one of the Clay Millennium Prize problems, US\$1,000,000). Almost everyone believes **$P \neq NP$** : finding a solution really is harder than checking one (compare “composing a great symphony” with “recognizing that one is great”).

Nobody has been able to prove it.

5.3 “What we think the world looks like” (slide 11)

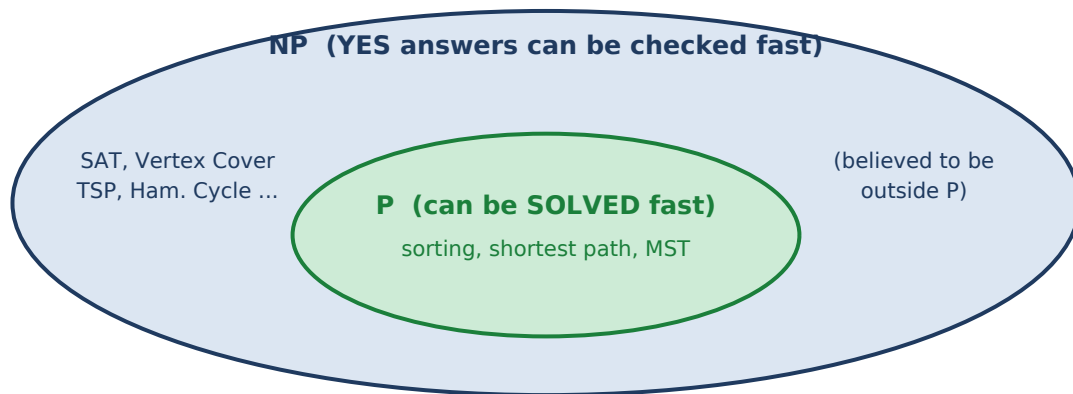


Figure 1 · The widely believed picture (assuming $P \neq NP$): P is a proper part of NP .

WATCH OUT · Do not confuse ‘believed’ with ‘proved’

In an exam, never write “ $P \neq NP$ ” as a fact. Write “assuming $P \neq NP$ ” or “it is widely believed that $P \neq NP$ ”.

Part 6 · Complement of a problem and co-NP (slides 12–16)

6.1 The complement X' (slide 12)

For every decision problem X there is a natural **complementary problem X'** : **YES and NO are swapped**. If the answer to X is YES then the answer to X' is NO (and vice versa).

Problem X	Complement X'
SAT: is the CNF formula satisfiable?	UNSAT: is the CNF formula unsatisfiable ?
Is graph G connected?	Is G disconnected?
Is n composite?	Is n prime?

6.2 P is closed under complement (slide 13)

KEY IDEA · $X \in P \Rightarrow X' \in P$

We have an efficient algorithm A for X . Define A' for X' : **run A and flip its answer**. Same running time, so $X' \in P$. ■

6.3 Is NP closed under complement? (slide 14)

SLIDE QUESTION (slide 14): Is it true that $X \in NP \Rightarrow X' \in NP$?

Not known (and believed false). With a certifier for X , you cannot just flip the answer. A certifier for X says YES if **there exists** a good certificate. Flipping would mean $X' = \text{YES}$ iff **for all** certificates the certifier rejects — and checking “for all” means trying exponentially many certificates. In the slide’s words: *YES-answer verification involves the “there exist” quantifier, whereas NO-answer verification involves “for all”*. Having an efficient certifier for X does not give you one for X' .

6.4 co-NP (slides 15-16)

KEY IDEA · Definition

co-NP = set of decision problems with a polynomial-time **verification for a NO answer**.
Equivalently: $X' \in \text{NP} \Leftrightarrow X \in \text{co-NP}$. (“Just the opposite of NP.”)

EXAMPLE · UNSAT $\in$ co-NP

The complement of UNSAT is SAT, and $\text{SAT} \in \text{NP}$, so $\text{UNSAT} \in \text{co-NP}$. Concretely: the NO answer to UNSAT means “the formula *is* satisfiable”, and a satisfying assignment is a short, checkable certificate.

WATCH OUT · co-NP is NOT “the problems outside NP”

co-NP is the set of *complements* of NP problems. It overlaps a lot with NP (for instance all of P lies in both).

EXAMPLE · PRIME (slide 16)

PRIME: is the number x prime? A NO answer means “ x is composite”, and a certificate is a divisor d with $1 < d < x$; verifying is just checking $x \bmod d = 0$. So **PRIME $\in$ co-NP**.

The slide says verifying a YES (“ x is prime”) does not *seem* easy — true at first sight. Historical footnotes: Pratt (1975) later showed short certificates for primality exist too, so $\text{PRIME} \in \text{NP} \cap \text{co-NP}$; and in 2002 Agrawal, Kayal and Saxena (AKS) proved **PRIME $\in$ P**. Lesson: a problem that looks hard can secretly be easy.

Part 7 · NP vs co-NP and how P, NP, co-NP relate (slides 17-21)

7.1 Two more questions (slides 17-18)

SLIDE QUESTION (slide 17): Does $\text{NP} = \text{co-NP}$?

We don’t know yet. The common belief is $\text{NP} \neq \text{co-NP}$ (slide 20).

SLIDE QUESTION (slide 17-18): What is the relationship between P and co-NP?

$P \subseteq \text{co-NP}$. Proof chain (slide 18): $X \in P \Rightarrow X' \in P$ (P closed under complement) $\Rightarrow X' \in \text{NP}$ ($P \subseteq \text{NP}$) $\Rightarrow X \in \text{co-NP}$ (definition of co-NP).

In words: if you can *solve* the problem fast, you can certainly verify a NO answer fast, without even looking at the certificate.

7.2 The full picture (slide 19)

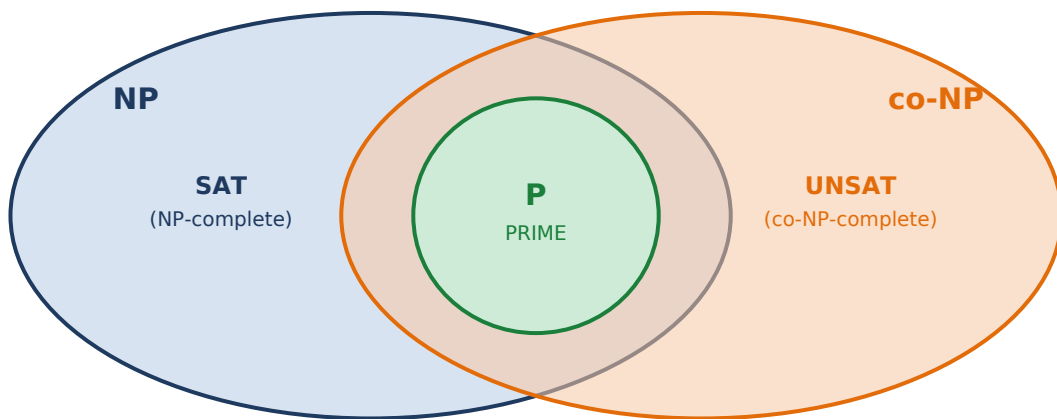


Figure 2 · Believed picture: P sits inside $NP \cap co-NP$; SAT is on the NP side, UNSAT on the co-NP side.

7.3 The belief and its consequences (slides 20-21)

Just as for P vs NP , the widespread belief is $NP \neq co-NP$, but it is unproven. And the slide adds: *proving $NP \neq co-NP$ is an even bigger step than proving $P \neq NP$. Why? Because of the implication below: $NP \neq co-NP$ is a **stronger** statement that **would give $P \neq NP$ for free**.*

KEY IDEA · $NP \neq co-NP \Rightarrow P \neq NP$ (equivalently $P = NP \Rightarrow NP = co-NP$)

These two are **contrapositives** of each other (" $A \Rightarrow B$ " is the same as " $\text{not } B \Rightarrow \text{not } A$ "), so proving one proves the other. Proof of the second form (slide 21): assume $P = NP$.

$NP \subseteq co-NP$: $X \in NP \Rightarrow X \in P$ (since $NP = P$) $\Rightarrow X' \in P$ (closure) $\Rightarrow X' \in NP$ ($P \subseteq NP$) $\Rightarrow X \in co-NP$.

$co-NP \subseteq NP$: $X \in co-NP \Rightarrow X' \in NP \Rightarrow X' \in P$ (since $NP = P$) $\Rightarrow X \in P$ (closure) $\Rightarrow X \in NP$.

Both inclusions $\Rightarrow NP = co-NP$. Slide's one-liner: " P is closed under complementation, so if $P = NP$ then $NP = P = \text{complement of } P = co-NP$." ■

Statement	Status
$P \subseteq NP$; $P \subseteq co-NP$	Proven
$P = NP$?	Open (believed false)
$NP = co-NP$?	Open (believed false)
$P = NP \Rightarrow NP = co-NP$	Proven implication
$NP \neq co-NP \Rightarrow P \neq NP$	Proven implication (contrapositive)
$NP = co-NP \Rightarrow P = NP$?	Not known — it does not follow

Part 8 · Polynomial-time reductions (slides 22-24)

8.1 The idea

A **reduction** is “solving problem A by using a solver for problem B”. Everyday version: to go from Dhaka to Chittagong you reduce the problem to “find the route in a map app” and then read off the answer. The slides write $A \leq_p B$ and say “Problem A is polynomial-time reducible to problem B”.

EXAMPLE · $\text{FIND-MAX} \leq_p \text{SORT}$

Problem A = “find the maximum of a list”. Problem B = “sort a list”. Given a list I : send it unchanged to a sorting black box (this is f), then take the **last element** of the sorted output (this is h). So max is solved using sorting, hence $\text{FIND-MAX} \leq_p \text{SORT}$.

8.2 Formal picture (slide 22)

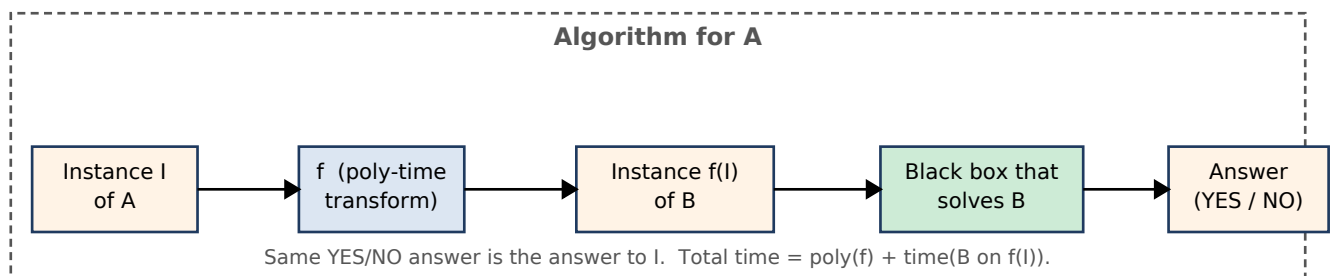


Figure 3 · Slide 22: an algorithm for A built from a transformation f and a black box for B (h maps solutions back when we need the actual solution, not just YES/NO).

For decision problems, the usual definition is: $A \leq_p B$ if there is a polynomial-time function f that maps every instance I of A to an instance $f(I)$ of B such that **I is a YES-instance of A $\Leftrightarrow f(I)$ is a YES-instance of B**. Both directions ($\Rightarrow$ and $\Leftarrow$) must be proved when you design a reduction.

8.3 Who is harder? (slide 23)

SLIDE QUESTION (slide 23): “A is at least as hard as B” or “B is at least as hard as A”? (the slide ticks the second)

B is at least as hard as A. If $A \leq_p B$, then B is strong enough to solve A (plus a little polynomial work). So B can be no easier than A. **Memory trick:** read $\leq$ like the number sign: $A \leq B$, so the problem on the right is the bigger (harder) one. The first statement (A at least as hard as B) is the wrong one, and it is a very common exam mistake.

8.4 What reductions let us conclude (slide 24)

If $A \leq_p B$, then...	Meaning
If you can solve B in polynomial time, then you can solve A in polynomial time.	Efficiency flows from B to A: run f (poly), then B (poly).
If you cannot solve A in polynomial time, then you cannot solve B in polynomial time.	Difficulty flows from A to B: it is the contrapositive of the sentence above.

The slide’s summary: **Difficulty flows from A to B; efficiency flows from B to A.** This is why, to show a *new* problem is hard, we reduce a *known hard* problem **to** the new one.

Transitivity (a useful tool)

If $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$: apply f_1 ($A \rightarrow B$) then f_2 ($B \rightarrow C$). If f_1 takes $O(n^a)$ time then its output has size at most $O(n^a)$, and f_2 runs in $O((n^a)^b) = O(n^{ab})$, still a polynomial. This is what allows *chains* of reductions in Part 9.

Part 9 · NP-hard, NP-complete, and how to prove it (slides 25–28)

9.1 NP-hard (slide 25)

KEY IDEA · Definition

Problem X is **NP-hard** if for **all** $Y \in \text{NP}$, $Y \leq_p X$. In words: a polynomial-time algorithm for X would give a polynomial-time algorithm for **every problem in NP**.

So an NP-hard problem is “at least as hard as everything in NP”. Notice: **X does not have to be in NP**. NP-hard problems can be even harder than NP (e.g. the Halting problem, which is not even decidable).

9.2 NP-complete (slide 26)

KEY IDEA · Definition

Problem X is **NP-complete** if (i) $X \in \text{NP}$ **and** (ii) for all $Y \in \text{NP}$, $Y \leq_p X$ (i.e. X is NP-hard). So **NP-complete = NP $\cap$ NP-hard**.

- NP-complete problems are the **hardest problems in NP**.
- They form a set of problems with the **same level of difficulty**: any two NP-complete problems reduce to each other.
- **Consequence 1**: if *any* NP-complete problem has a polynomial algorithm, then **$P = \text{NP}$** (all of NP collapses into P).
- **Consequence 2**: if $P \neq \text{NP}$, then *no* NP-complete problem has a polynomial algorithm.
- **Practical meaning**: if your problem is NP-complete, stop hunting for a perfect fast algorithm. Use approximation, heuristics, restrict to special cases, or small inputs.

9.3 The picture (slide 27)

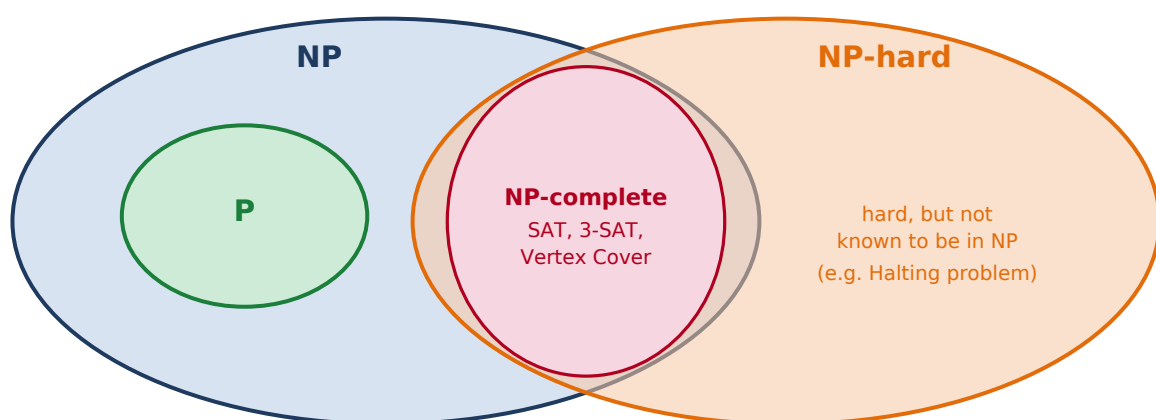


Figure 4 · Assuming $P \neq \text{NP}$: NP-complete is the overlap of NP and NP-hard; P is inside NP but away from the NP-complete region.

Class	Must be in NP?	Meaning
P	Yes	Solvable in polynomial time
NP	—	YES answers verifiable in polynomial time
NP-hard	No	At least as hard as every NP problem
NP-complete	Yes	In NP and NP-hard: the hardest problems of NP

9.4 How do we prove a problem X is NP-complete? (slide 28)

Direct way: prove $Y \leq_p X$ for every $Y \in \text{NP}$. That is a huge task (there are infinitely many Y). It was done **once**, by Cook and Levin (1971), for SAT: **SAT is NP-complete**.

Alternative (slide 28): prove that a **known NP-complete problem Y** reduces to X: $Y \leq_p X$. Why is that enough? For any $Z \in \text{NP}$ we have $Z \leq_p Y$ (since Y is NP-complete) and $Y \leq_p X$, so by transitivity $Z \leq_p X$. That is exactly NP-hardness of X.

KEY IDEA · Recipe for an NP-completeness proof

Step 1. Show $X \in \text{NP}$: describe a certificate and a polynomial-time verifier.

Step 2. Choose a **known NP-complete** problem Y (SAT, 3-SAT, Independent Set, Vertex Cover, ...).

Step 3. Describe a polynomial-time transformation f from an instance of Y to an instance of X.

Step 4. Prove: Y-instance is YES $\Leftrightarrow$ f(instance) is YES for X. Prove both directions ($\Rightarrow$ and $\Leftarrow$).

Step 5. Argue f runs in polynomial time. Conclude: X is NP-complete.

WATCH OUT · The direction mistake (the lecture's own warning)

Reduce the **known NP-complete problem TO the new problem** ($Y \leq_p X$), not the other way round. Showing $X \leq_p Y$ only says “X is no harder than Y”, which tells you nothing about X being hard. Also do not forget Step 1 ($X \in \text{NP}$); the deck's last slide reminds you: “Do not forget to prove that TSP is in NP”.

What comes next in the lecture (slides 29+): the chain of reductions 3-SAT $\rightarrow$ Independent Set $\rightarrow$ Vertex Cover $\rightarrow$ Set Cover, and Hamiltonian Cycle $\rightarrow$ TSP. These are worked examples of the recipe above.

Part 10 · One-page cheat sheet

Term	Definition (short)
Decision problem	Output is YES/NO.
P	Decision problems solvable in polynomial time.
NP	Decision problems whose YES answers can be verified in polynomial time given a certificate.
co-NP	Decision problems whose NO answers can be verified in polynomial time. ($X \in \text{co-NP} \Leftrightarrow X' \in \text{NP}$)
Complement X'	Same instances, YES $\Leftrightarrow$ NO.
$A \leq_p B$	A reduces to B in polynomial time: B is at least as hard as A.
NP-hard	All $Y \in \text{NP}$ satisfy $Y \leq_p X$. (May be outside NP.)
NP-complete	In NP and NP-hard.

Fact	Why
$P \subseteq \text{NP}$	Certifier ignores the certificate and solves the problem.
P closed under complement	Run the algorithm, flip the answer.
$P \subseteq \text{co-NP}$	$X \in P \Rightarrow X' \in P \Rightarrow X' \in \text{NP} \Rightarrow X \in \text{co-NP}$.
$P = \text{NP} \Rightarrow \text{NP} = \text{co-NP}$	P is closed under complement.
$A \leq_p B, B \in P \Rightarrow A \in P$	Efficiency flows from B to A.
$A \leq_p B, A \notin P \Rightarrow B \notin P$	Difficulty flows from A to B.
$A \leq_p B, B \leq_p C \Rightarrow A \leq_p C$	Transitivity (compose f_1 and f_2).
$Y \text{ NPC}, Y \leq_p X, X \in \text{NP} \Rightarrow X \text{ NPC}$	Standard proof method.
Any NPC problem in $P \Rightarrow P = \text{NP}$	Every NP problem reduces to it.
Any NPC problem in $\text{co-NP} \Rightarrow \text{NP} = \text{co-NP}$	See Q26 in Part 11.

REMEMBER · Exam habits

Always say “certificate + polynomial verifier” when proving membership in NP. Always draw the arrow of your reduction: **known-hard** $\rightarrow$ **new problem**. Say “assuming $P \neq \text{NP}$ ” whenever you use the belief. For an NP-completeness proof write all five steps of the recipe.

Part 11 · CT practice questions (Easy → Hard)

Try to answer each question on paper first. Answers are in Part 12 with the same numbers. Marks in an actual CT: definitions 1-2 marks, proofs 3-5 marks.

Level 1 · Easy (definitions, 1-2 marks)

- Q1.** What does NP stand for? Does it mean “not polynomial”?
- Q2.** State True or False and give a one-line reason: (a) $P \subseteq NP$. (b) If $X \in P$ then the complement $X' \in P$. (c) NP-complete problems are the easiest problems in NP. (d) If $A \leq_p B$ then A is at least as hard as B. (e) It has been proved that $P \neq NP$.
- Q3.** Classify each as a decision problem or an optimization problem: (i) find a minimum spanning tree; (ii) is there a spanning tree of weight ≤ 50 ? (iii) is a given CNF formula satisfiable? (iv) find the shortest path from s to t.
- Q4.** Write the complement problem of: (a) SAT; (b) “Is graph G connected?”; (c) “Is n a composite number?”
- Q5.** Consider the formula $(a \vee b \vee c) \wedge (a' \vee b' \vee c) \wedge (a \vee b' \vee c')$. Verify that $a = 1, b = 0, c = 1$ satisfies it. What is such an assignment called and what does it prove?
- Q6.** Fill in the blanks: $A \leq_p B$ means ____ is at least as hard as _____. If $B \in P$ and $A \leq_p B$ then $A \in$ _____.
- Q7.** Which of these running times are polynomial: $n^3, 2^n, n \log n, n^{50}, n!, 1.01^n$?

Level 2 · Medium (apply the definitions)

- Q8.** Write the decision versions of (a) Traveling Salesman (find the shortest tour), (b) Maximum Clique, (c) Minimum Spanning Tree. Why is the bound needed?
- Q9.** Show that “Given a weighted graph G, vertices s, t and a number x: is there an s-t path of length $\leq x$?” is in NP. Give the certificate, the verifier and its running time.
- Q10.** Show that $SAT \in NP$. Give the certificate and the running time of the verifier for a formula with n variables and m clauses of at most k literals each.
- Q11.** Why is a YES answer to SAT easy to verify but a NO answer apparently not? In which class does UNSAT lie?
- Q12.** Show that the problem COMPOSITE (“is n composite?”) is in NP. What does this say about PRIME?
- Q13.** Prove that $P \subseteq NP$.
- Q14.** Prove that P is closed under complement.
- Q15.** Show that every problem in NP can be solved in exponential time.
- Q16.** Explain the difference between NP-hard and NP-complete. Give an example of a problem that is NP-hard but not known to be in NP.

Level 3 · Medium-hard (reasoning with reductions and classes)

- Q17.** Prove transitivity: if $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$.
- Q18.** Let $A \leq_p B$. Which conclusions are valid? (a) $B \in P \Rightarrow A \in P$. (b) $A \notin P \Rightarrow B \notin P$. (c) $A \text{ NP-hard} \Rightarrow B \text{ NP-hard}$. (d) $B \text{ NP-hard} \Rightarrow A \text{ NP-hard}$. (e) $A \text{ NP-complete and } B \in NP \Rightarrow B \text{ NP-complete}$.
- Q19.** A student wants to prove that problem X is NP-complete. He shows $X \in NP$ and gives a polynomial reduction $X \leq_p 3\text{-SAT}$. Has he proved it? What has he actually shown and what should he do?

- Q20.** Assume you have a polynomial-time black box for the decision version of shortest path. Describe how to find the length of a shortest s-t path and the path itself, and count the calls.
- Q21.** Give a reduction $\text{FIND-MAX} \leq_p \text{SORT}$. Identify f and h. Does it prove that finding the maximum is hard?
- Q22.** Why can't we show " $X \in \text{NP} \Rightarrow X' \in \text{NP}$ " just by flipping the answer of the certifier, as we did for P?
- Q23.** Prove $P \subseteq \text{co-NP}$ using a chain of implications.
- Q24.** Show that $\text{NP} \neq \text{co-NP}$ implies $P \neq \text{NP}$. Explain why proving $\text{NP} \neq \text{co-NP}$ would be a bigger achievement than proving $P \neq \text{NP}$.

Level 4 · Hard (proofs and implications)

- Q25.** Prove: if $P = \text{NP}$ then $\text{NP} = \text{co-NP}$. Is the converse known to be true?
- Q26.** Prove: if some NP-complete problem Y is in co-NP, then $\text{NP} = \text{co-NP}$.
- Q27.** Suppose someone proves $\text{UNSAT} \in P$. What follows? Explain each step.
- Q28.** Suppose you have a black box that decides SAT (yes/no). Show how to **find** a satisfying assignment with $n + 1$ calls (n variables).
- Q29.** Prove: if Y is NP-complete, $X \in \text{NP}$ and $Y \leq_p X$, then X is NP-complete.
- Q30.** A researcher claims a polynomial-time reduction from SAT to the shortest-path decision problem ($\text{SAT} \leq_p \text{SHORTEST-PATH}$). What would this imply and what should you conclude?
- Q31.** Assuming $\text{NP} \neq \text{co-NP}$, can UNSAT be NP-complete? Justify.
- Q32.** (Challenge) Is the following proof correct? "Since every problem in NP reduces to SAT and $\text{SAT} \in \text{NP}$, if we show $\text{SAT} \leq_p X$ then X is NP-complete."
- Q33.** (Challenge) An algorithm takes 2^n steps on an input of size n. A computer performs 10^9 steps per second. Estimate the running time for $n = 40$ and $n = 60$, and explain what this suggests about "efficient".

Part 12 · Answers

Level 1 · Easy (definitions, 1-2 marks)

Q1. What does NP stand for? Does it mean “not polynomial”?

Answer 1

NP stands for **Nondeterministic Polynomial time**. It does **not** mean “not polynomial”. NP is the set of decision problems whose YES answers can be verified in polynomial time given a certificate. Every problem in P is in NP, so NP cannot mean “not polynomial”.

Q2. State True or False and give a one-line reason: (a) $P \subseteq NP$. (b) If $X \in P$ then the complement $X' \in P$. (c) NP-complete problems are the easiest problems in NP. (d) If $A \leq_p B$ then A is at least as hard as B. (e) It has been proved that $P \neq NP$.

Answer 2

- (a) **True.** A certifier can ignore the certificate and run the polynomial solver.
- (b) **True.** Run the algorithm for X and flip the answer.
- (c) **False.** They are the *hardest* problems in NP.
- (d) **False.** B is at least as hard as A.
- (e) **False.** It is an open problem (believed true but unproven).

Q3. Classify each as a decision problem or an optimization problem: (i) find a minimum spanning tree; (ii) is there a spanning tree of weight ≤ 50 ? (iii) is a given CNF formula satisfiable? (iv) find the shortest path from s to t.

Answer 3

- (i) optimization, (ii) decision, (iii) decision, (iv) optimization.

Q4. Write the complement problem of: (a) SAT; (b) “Is graph G connected?”; (c) “Is n a composite number?”

Answer 4

- (a) UNSAT: is the CNF formula unsatisfiable? (b) Is G disconnected? (c) Is n prime? (for $n \geq 2$). In each case YES and NO swap.

Q5. Consider the formula $(a \vee b \vee c) \wedge (a' \vee b' \vee c) \wedge (a \vee b' \vee c')$. Verify that $a = 1, b = 0, c = 1$ satisfies it. What is such an assignment called and what does it prove?

Answer 5

Clause 1: $a = 1 \Rightarrow \text{true}$. Clause 2: $a' = 0, b' = 1 \Rightarrow \text{true}$. Clause 3: $a = 1 \Rightarrow \text{true}$. All clauses are true, so the formula is satisfied.

The assignment is a **certificate** (witness). It proves the instance is a YES-instance of SAT, and checking it took time linear in the formula size, which is why $SAT \in NP$.

Q6. Fill in the blanks: $A \leq_p B$ means ____ is at least as hard as _____. If $B \in P$ and $A \leq_p B$ then $A \in$ _____.

Answer 6

B is at least as hard as A. If $B \in P$ then $A \in P$.

Q7. Which of these running times are polynomial: $n^3, 2^n, n \log n, n^{50}, n!, 1.01^n$?

Answer 7

Polynomial: **$n^3, n \log n (\leq n^2), n^{50}$** (impractical but polynomial). Not polynomial: $2^n, n!$, and 1.01^n (exponential; it starts small but eventually beats every polynomial).

Level 2 · Medium (apply the definitions)

Q8. Write the decision versions of (a) Traveling Salesman (find the shortest tour), (b) Maximum Clique, (c) Minimum Spanning Tree. Why is the bound needed?

Answer 8

(a) Is there a tour visiting every city exactly once with total length $\leq L$? (b) Does G contain a clique of size $\geq k$? (c) Is there a spanning tree of total weight $\leq W$?

The bound turns “find the best” into a yes/no question about “good enough”. Without it, the question would be trivial (a tour or a spanning tree almost always exists) and would say nothing about optimality.

Q9. Show that “Given a weighted graph G , vertices s, t and a number x : is there an s - t path of length $\leq x$?” is in NP. Give the certificate, the verifier and its running time.

Answer 9

Certificate: the path itself, a sequence of vertices $v_0 = s, v_1, \dots, v_m = t$ ($m < n$).

Verifier: check that $v_0 = s$ and $v_m = t$, that each consecutive pair (v_i, v_{i+1}) is an edge, and that the sum of their weights is $\leq x$. Time $O(n)$ with an adjacency matrix (polynomial).

Certificate length is polynomial in the input, the verifier is polynomial, so the problem is in NP. (It is also in P via Dijkstra.)

Q10. Show that $\text{SAT} \in \text{NP}$. Give the certificate and the running time of the verifier for a formula with n variables and m clauses of at most k literals each.

Answer 10

Certificate: a truth assignment for the n variables (n bits).

Verifier: substitute the values into every clause, evaluate each OR, then check that all m clauses are true. Time $O(m \cdot k)$, linear in the formula size. Hence $\text{SAT} \in \text{NP}$.

Q11. Why is a YES answer to SAT easy to verify but a NO answer apparently not? In which class does UNSAT lie?

Answer 11

YES has the form “**there exists** an assignment that works”, and one assignment is a short proof. NO has the form “**for all** 2^n assignments the formula is false”. No short proof for that is known.

$\text{UNSAT} \in \text{co-NP}$ (its complement SAT is in NP). Whether $\text{UNSAT} \in \text{NP}$ is unknown; it would imply $\text{NP} = \text{co-NP}$ (see Q26).

Q12. Show that the problem COMPOSITE (“is n composite?”) is in NP. What does this say about PRIME?

Answer 12

Certificate: a divisor d with $1 < d < n$. **Verifier:** compute $n \bmod d$ and check it is 0 (polynomial in the number of bits of n). So $\text{COMPOSITE} \in \text{NP}$.

PRIME is the complement of COMPOSITE, therefore **PRIME $\in$ co-NP**. (In fact $\text{PRIME} \in \text{P}$ by the AKS algorithm, so also in NP.)

Q13. Prove that $\text{P} \subseteq \text{NP}$.

Answer 13

Let $X \in \text{P}$ and let A solve X in polynomial time. Define a certifier $B(I, S)$ that ignores S and returns $A(I)$. B is polynomial and accepts exactly the YES instances, so $X \in \text{NP}$. ■

Q14. Prove that P is closed under complement.

Answer 14

Let A be a polynomial algorithm for X. Algorithm A': on input I, run A(I) and output the opposite answer. A' is correct for X' and has the same polynomial running time. ■

Q15. Show that every problem in NP can be solved in exponential time.

Answer 15

Let the certificate length be at most $p(n)$ for a polynomial p and the verifier time be $q(n)$. Try all $2^{p(n)}$ possible certificates; run the verifier on each; answer YES iff some certificate is accepted. Time: $2^{p(n)} \cdot q(n)$, which is exponential. Therefore $P \subseteq NP \subseteq EXP$ (exponential time).

Q16. Explain the difference between NP-hard and NP-complete. Give an example of a problem that is NP-hard but not known to be in NP.

Answer 16

NP-hard: every $Y \in NP$ reduces to X (X is at least as hard as all of NP); X need not be in NP.

NP-complete: X is NP-hard **and** $X \in NP$. So NP-complete = $NP \cap NP\text{-hard}$.

Example of NP-hard but not in NP: the **Halting problem** (undecidable, so certainly not in NP).

Optimization versions of NP-complete problems (e.g. "find the shortest TSP tour") are also NP-hard if reductions may call them as a subroutine, but they are not decision problems, hence not in NP.

Level 3 · Medium-hard (reasoning with reductions and classes)

Q17. Prove transitivity: if $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$.

Answer 17

Let f_1 map A-instances to B-instances in time $O(n^a)$, and f_2 map B-instances to C-instances in time $O(m^b)$ where m is the size of its input. Define $f(I) = f_2(f_1(I))$. $|f_1(I)| \leq O(n^a)$, so f_2 runs in $O(n^{ab})$ time; total polynomial. $I \in YES(A) \Leftrightarrow f_1(I) \in YES(B) \Leftrightarrow f_2(f_1(I)) \in YES(C)$. Hence $A \leq_p C$. ■

Q18. Let $A \leq_p B$. Which conclusions are valid? (a) $B \in P \Rightarrow A \in P$. (b) $A \notin P \Rightarrow B \notin P$. (c) $A \text{ NP-hard} \Rightarrow B \text{ NP-hard}$. (d) $B \text{ NP-hard} \Rightarrow A \text{ NP-hard}$. (e) $A \text{ NP-complete and } B \in NP \Rightarrow B \text{ NP-complete}$.

Answer 18

(a) **Valid** (efficiency flows $B \rightarrow A$). (b) **Valid** (contrapositive of (a); difficulty flows $A \rightarrow B$).

(c) **Valid**: for any $Y \in NP$, $Y \leq_p A \leq_p B$. (d) **Not valid**: every problem in P reduces to SAT (which is NP-hard), so (d) would make every problem in P NP-hard, which would force $P = NP$.

(e) **Valid**: $B \in NP$ is given and (c) gives B NP-hard.

Q19. A student wants to prove that problem X is NP-complete. He shows $X \in NP$ and gives a polynomial reduction $X \leq_p 3\text{-SAT}$. Has he proved it? What has he actually shown and what should he do?

Answer 19

No. The reduction is in the **wrong direction**. $X \leq_p 3\text{-SAT}$ says X is *no harder* than 3-SAT — it gives an upper bound on the difficulty of X (in fact it would prove $X \in NP$ again, since $3\text{-SAT} \in NP$). To prove X is hard he must show **3-SAT $\leq_p X$** (a known NP-complete problem reduces to X), and prove $X \in NP$.

Q20. Assume you have a polynomial-time black box for the decision version of shortest path. Describe how to find the length of a shortest s - t path and the path itself, and count the calls.

Answer 20

Let W be the sum of all (integer) edge weights. **Length:** binary search on $x \in [0, W]$ using $\text{DECIDE}(G, s, t, x)$; about $\log_2 W$ calls. Call the result L .

Path: for each edge e ($|E|$ edges): remove e ; if $\text{DECIDE}(G - e, s, t, L)$ is NO, put e back. The edges that remain form a shortest path. That is $|E|$ more calls. Total $O(\log W + |E|)$ calls, polynomial. (See Part 3.)

Q21. Give a reduction $\text{FIND-MAX} \leq_p \text{SORT}$. Identify f and h . Does it prove that finding the maximum is hard?

Answer 21

f = identity (pass the list to the sorter); h = return the last element of the sorted list. Cost: $O(1)$ plus the sort.

It proves nothing about FIND-MAX being hard. It says SORT is *at least as hard as* FIND-MAX. Difficulty flows from the easier problem A to B only if A is hard, and FIND-MAX is $O(n)$.

Q22. Why can't we show " $X \in \text{NP} \Rightarrow X' \in \text{NP}$ " just by flipping the answer of the certifier, as we did for P?

Answer 22

A certifier for X answers a question of the form "is *this* S a valid certificate?". Flipping its output says " S is *not* a certificate", which does not show that **no** certificate exists. $X' = \text{YES}$ exactly when **all** certificates fail, which is a "for all" statement; checking it directly means trying exponentially many certificates. For an algorithm that solves X (class P) there is no certificate, so flipping is legitimate.

Q23. Prove $P \subseteq \text{co-NP}$ using a chain of implications.

Answer 23

$X \in P \Rightarrow X' \in P$ (P closed under complement) $\Rightarrow X' \in \text{NP}$ ($P \subseteq \text{NP}$) $\Rightarrow X \in \text{co-NP}$ (definition: $X' \in \text{NP}$ means $X \in \text{co-NP}$). ■

Q24. Show that $\text{NP} \neq \text{co-NP}$ implies $P \neq \text{NP}$. Explain why proving $\text{NP} \neq \text{co-NP}$ would be a bigger achievement than proving $P \neq \text{NP}$.

Answer 24

We prove the contrapositive: $P = \text{NP} \Rightarrow \text{NP} = \text{co-NP}$. (Full proof in Q25.) The contrapositive gives: $\text{NP} \neq \text{co-NP} \Rightarrow P \neq \text{NP}$.

So a proof of $\text{NP} \neq \text{co-NP}$ would automatically give $P \neq \text{NP}$ (plus more). The reverse is not known to hold, so $\text{NP} \neq \text{co-NP}$ is the stronger statement.

Level 4 · Hard (proofs and implications)

Q25. Prove: if $P = \text{NP}$ then $\text{NP} = \text{co-NP}$. Is the converse known to be true?

Answer 25

Assume $P = \text{NP}$.

$\text{NP} \subseteq \text{co-NP}$: $X \in \text{NP} \Rightarrow X \in P \Rightarrow X' \in P \Rightarrow X' \in \text{NP} \Rightarrow X \in \text{co-NP}$.

$\text{co-NP} \subseteq \text{NP}$: $X \in \text{co-NP} \Rightarrow X' \in \text{NP} \Rightarrow X' \in P \Rightarrow X \in P \Rightarrow X \in \text{NP}$. Hence $\text{NP} = \text{co-NP}$. ■

The converse ($\text{NP} = \text{co-NP} \Rightarrow P = \text{NP}$) is **not known** to hold.

Q26. Prove: if some NP-complete problem Y is in co-NP, then $NP = co-NP$.

Answer 26

Let $L \in NP$ be arbitrary. Since Y is NP-complete, $L \leq_p Y$ via a function $f: x \in L \Leftrightarrow f(x) \in Y$. Taking complements, $x \in L' \Leftrightarrow f(x) \in Y'$. So $L' \leq_p Y'$.

$Y \in co-NP$ means $Y' \in NP$. NP is closed under polynomial reductions (given x , compute $f(x)$ and run the NP certifier of Y' on $f(x)$), therefore $L' \in NP$, i.e. $L \in co-NP$. This shows $NP \subseteq co-NP$.

Conversely, if $L \in co-NP$ then $L' \in NP \subseteq co-NP$, so $L'' = L \in NP$. Hence $co-NP \subseteq NP$. Together $NP = co-NP$. ■

Q27. Suppose someone proves $UNSAT \in P$. What follows? Explain each step.

Answer 27

$UNSAT \in P \Rightarrow (P \text{ closed under complement}) SAT \in P$. SAT is NP-complete (Cook-Levin), so every problem in NP reduces to SAT and therefore is in P: $NP \subseteq P$, i.e. **$P = NP$** . By Q25 also $NP = co-NP$.

Q28. Suppose you have a black box that decides SAT (yes/no). Show how to **find** a satisfying assignment with $n + 1$ calls (n variables).

Answer 28

Call the black box on F . If NO, output “unsatisfiable”. Otherwise for $i = 1..n$: set $x_i = 1$ in F (simplify the formula). If the black box says the simplified formula is satisfiable, keep $x_i = 1$; else it must be satisfiable with $x_i = 0$, so set $x_i = 0$. After n steps every variable has a value and the formula is still satisfiable. This is the “search reduces to decision” idea from slide 8 for SAT.

Q29. Prove: if Y is NP-complete, $X \in NP$ and $Y \leq_p X$, then X is NP-complete.

Answer 29

$X \in NP$ is given. For NP-hardness take any $Z \in NP$. Since Y is NP-complete, $Z \leq_p Y$. With $Y \leq_p X$ and transitivity (Q17) we get $Z \leq_p X$. As Z was arbitrary, X is NP-hard; combined with $X \in NP$, X is NP-complete. ■

Q30. A researcher claims a polynomial-time reduction from SAT to the shortest-path decision problem ($SAT \leq_p \text{SHORTEST-PATH}$). What would this imply and what should you conclude?

Answer 30

$\text{SHORTEST-PATH} \in P$ (Dijkstra). By “efficiency flows from B to A”, $SAT \in P$; since SAT is NP-complete, **$P = NP$** . As nobody believes $P = NP$, the claim should be treated with great suspicion: either the reduction is wrong (for example it is not polynomial, or the $YES \Leftrightarrow YES$ property fails) or it would be a revolutionary result.

Q31. Assuming $NP \neq co-NP$, can UNSAT be NP-complete? Justify.

Answer 31

No. If UNSAT were NP-complete then $UNSAT \in NP$, so its complement $SAT \in co-NP$. SAT is an NP-complete problem in co-NP, hence $NP = co-NP$ by Q26, contradicting the assumption. (UNSAT is instead **co-NP-complete**.)

Q32. (Challenge) Is the following proof correct? “Since every problem in NP reduces to SAT and $SAT \in NP$, if we show $SAT \leq_p X$ then X is NP-complete.”

Answer 32

Not quite: the argument shows X is **NP-hard** ($SAT \leq_p X$ and transitivity), but X might be outside NP, so you must **also prove $X \in NP$** (certificate + verifier). Without it you can only claim NP-hardness.

Q33. (Challenge) An algorithm takes 2^n steps on an input of size n . A computer performs 10^9 steps per second. Estimate the running time for $n = 40$ and $n = 60$, and explain what this suggests about “efficient”.

Answer 33

$n = 40$: $2^{40} \approx 1.1 \times 10^{12}$ steps $\approx 1,100$ s ≈ 18 minutes. $n = 60$: $2^{60} \approx 1.15 \times 10^{18}$ steps $\approx 1.15 \times 10^9$ s ≈ 36 years. Each +10 in n multiplies the time by about 1,000.

So an exponential algorithm becomes unusable for moderately sized inputs even on very fast machines, whereas a polynomial algorithm scales gracefully. That is why “efficient” is defined as polynomial time.

End of guide. The remaining slides (29 onward: Independent Set, Vertex Cover, Set Cover, 3-SAT $\rightarrow$ Independent Set, Ham-Cycle $\rightarrow$ TSP) build directly on this material.